ECC Assignment - 2

Spark Installation:

Step 1: Added rules & Port Allocation

```
exouser@assign2vp:~$ sudo ufw allow ssh
Skipping adding existing rule
Skipping adding existing rule (v6)
exouser@assign2vp:~$ sudo ufw allow 49524/tcp
Rule added
Rule added (v6)
exouser@assign2vp:~$ sudo ufw allow from any to 172.17.0.1 port 5901 proto tcp
Skipping adding existing rule
exouser@assign2vp:~$ sudo apt update && upgrade
Get:1 https://nvidia.github.io/libnvidia-container/stable/ubuntu18.04/amd64  InRelease [1484 B]
Hit:2 https://nvidia.github.io/nvidia-container-runtime/stable/ubuntu18.04/amd64  InRelease
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Hit:4 https://nvidia.github.io/nvidia-docker/ubuntu18.04/amd64  InRelease
Hit:5 https://download.ceph.com/debian-pacific focal InRelease
Hit:6 http://ppa.launchpad.net/mozillateam/ppa/ubuntu jammy InRelease
Hit:7 http://nova.clouds.archive.ubuntu.com/ubuntu jammy InRelease
Get:8 http://nova.clouds.archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Hit:9 http://nova.clouds.archive.ubuntu.com/ubuntu jammy-backports InRelease
Fetched 231 kB in 1s (254 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
```

Step 2: Installed jdk and python

```
exouser@assign2vp:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
exouser@assign2vp:~$ sudo apt-get install openjdk-11-jre
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openjdk-11-jre is already the newest version (11.0.22+7-0ubuntu2~22.04.1).
openjdk-11-jre set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
exouser@assign2vp:~$ sudo apt-get install openjdk-11-jdk
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openjdk-11-jdk is already the newest version (11.0.22+7-0ubuntu2~22.04.1).
openjdk-11-jdk set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
exouser@assign2vp:~$ sudo apt install python3 python3-pip ipython3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.10.6-1~22.04).
python3 set to manually installed.
python3-pip is already the newest version (22.0.2+dfsg-1ubuntu0.4).
The following additional packages will be installed:
  blt fonts-lyx libblas3 libboost-dev libboost1.74-dev libjs-jquery-ui
  liblapack3 liblbfgsb0 libopenblas-dev libopenblas-pthread-dev libopenblas0
```

```
Setting up python3-sympy (1.9-1) ...
Setting up tk8.6-blt2.5 (2.5.3+dfsg-4.1build2) ...
Setting up liblapack3:amd64 (3.10.0-2ubuntu1) ...
Setting up python3-jedi (0.18.0-1) ...
Setting up blt (2.5.3+dfsg-4.1build2) ...
Setting up python3-tk:amd64 (3.10.8-1~22.04) ...
Setting up python3-bs4 (4.10.0-2) ...
Setting up python3-matplotlib-inline (0.1.3-1) ...
Setting up python3-fs (2.4.12-1) ...
Setting up libopenblas-dev:amd64 (0.3.20+ds-1) ...
Setting up python3-pil.imagetk:amd64 (9.0.1-1ubuntu0.2) ...
Setting up python3-ipython (7.31.1-1) ...
Setting up python3-pythran (0.10.0+ds2-1) ...
Setting up ipython3 (7.31.1-1) ...
Setting up python3-scipy (1.8.0-1exp2ubuntu1) ...
Setting up python3-fonttools (4.29.1-2build1) ...
Setting up python3-ufolib2 (0.13.1+dfsg1-1) ...
Setting up python3-matplotlib (3.5.1-2build1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.6) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.
```

Step 3: Installed PySpark in Jupyter:

```
exouser@assign2vp:~$ pip3 install jupyter py4j pyspark
Defaulting to user installation because normal site-packages is not writeable
Collecting jupyter
  Downloading jupyter-1.0.0-py2.py3-none-any.whl (2.7 kB)
Collecting py4j
  Downloading py4j-0.10.9.7-py2.py3-none-any.whl (200 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 200.5/200.5 KB 5.3 MB/s eta 0:00:00
Collecting pyspark
  Downloading pyspark-3.5.1.tar.gz (317.0 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 170.4/317.0 MB 29.8 MB/s eta 0:00:05
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 213.1/317.0 MB 38.0 MB/s eta 0:00:03
317.0/317.0 MB 7.8 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Collecting ipywidgets
  Downloading ipywidgets-8.1.2-py3-none-any.whl (139 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 139.4/139.4 KB 33.7 MB/s eta 0:00:00
Collecting qtconsole
  Downloading qtconsole-5.5.1-py3-none-any.whl (123 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 123.4/123.4 KB 31.1 MB/s eta 0:00:00
Collecting ipykernel
  Downloading ipykernel-6.29.4-py3-none-any.whl (117 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 117.1/117.1 KB 29.6 MB/s eta 0:00:00
Collecting jupyter-console
  Downloading jupyter_console-6.6.3-py3-none-any.whl (24 kB)
Collecting notebook
  Downloading notebook-7.1.2-py3-none-any.whl (5.0 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.0/5.0 MB 27.1 MB/s eta 0:00:00
Collecting nbconvert
  Downloading nbconvert-7.16.3-py3-none-any.whl (257 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 257.4/257.4 KB 40.2 MB/s eta 0:00:00
Collecting pyzmq>=24
```

Step 4: Installed Scala

```
exouser@assign2vp:~$ vim .bashrc
exouser@assign2vp:~$ source .bashrc
exouser@assign2vp:~$ wget https://downloads.lightbend.com/scala/2.13.3/scala-2.13.3.tgz
--2024-04-09 18:59:43--  https://downloads.lightbend.com/scala/2.13.3/scala-2.13.3.tgz
Resolving downloads.lightbend.com (downloads.lightbend.com)... 52.84.125.74, 52.84.125.77, 52.84.125.87
, ...
Connecting to downloads.lightbend.com (downloads.lightbend.com)|52.84.125.74|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 22414008 (21M) [application/octet-stream]
Saving to: 'scala-2.13.3.tgz'

scala-2.13.3.tgz    100%[===================>]  21.38M  77.6MB/s    in 0.3s

2024-04-09 18:59:43 (77.6 MB/s) - 'scala-2.13.3.tgz' saved [22414008/22414008]
```

Step 5: Made changes and added the Path in the bashrc File.

```
alias jupyter-ntebook='~/.local/bin/jupyter-notebook --no-browser'

export SCALA_HOME=/home/exouser/scala-2.13.3
export PATH=$PATH:$SCALA_HOME/bin

export SPARK_HOME=/home/exouser/spark-3.1.1-bin-hadoop3.2
export PATH=$PATH:$SPARK_HOME/bin
export PYSPARK_PYTHON=/home/exouser/python
".bashrc" 126L, 4026B                                         126,35        Bo
```

Step 6: Started Spark+Hadoop

```
exouser@assign2vp:~$ cd $SPARK_HOME
exouser@assign2vp:~/spark-3.1.1-bin-hadoop3.2$ ./sbin/start-master.sh
starting org.apache.spark.deploy.master.Master, logging to /home/exouser/spark-3.1.1-bin-hadoop3.2/logs
/spark-exouser-org.apache.spark.deploy.master.Master-1-assign2vp.out
exouser@assign2vp:~/spark-3.1.1-bin-hadoop3.2$ ./sbin/start-worker.sh spark://ubuntu1:7077
starting org.apache.spark.deploy.worker.Worker, logging to /home/exouser/spark-3.1.1-bin-hadoop3.2/logs
/spark-exouser-org.apache.spark.deploy.worker.Worker-1-assign2vp.out
```

```
exouser@assign2vp:~$ tar xvf spark-3.1.1-bin-hadoop3.2.tgz
spark-3.1.1-bin-hadoop3.2/
spark-3.1.1-bin-hadoop3.2/NOTICE
spark-3.1.1-bin-hadoop3.2/kubernetes/
spark-3.1.1-bin-hadoop3.2/kubernetes/tests/
spark-3.1.1-bin-hadoop3.2/kubernetes/tests/python_executable_check.py
spark-3.1.1-bin-hadoop3.2/kubernetes/tests/autoscale.py
spark-3.1.1-bin-hadoop3.2/kubernetes/tests/worker_memory_check.py
spark-3.1.1-bin-hadoop3.2/kubernetes/tests/py_container_checks.py
spark-3.1.1-bin-hadoop3.2/kubernetes/tests/decommissioning.py
spark-3.1.1-bin-hadoop3.2/kubernetes/tests/pyfiles.py
spark-3.1.1-bin-hadoop3.2/kubernetes/tests/decommissioning_cleanup.py
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/spark/
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/spark/decom.sh
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/spark/entrypoint.sh
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/spark/bindings/
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/spark/bindings/R/
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/spark/bindings/R/Dockerfile
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/spark/bindings/python/
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/spark/bindings/python/Dockerfile
spark-3.1.1-bin-hadoop3.2/kubernetes/dockerfiles/spark/Dockerfile
spark-3.1.1-bin-hadoop3.2/jars/
spark-3.1.1-bin-hadoop3.2/jars/RoaringBitmap-0.9.0.jar
```

Step 7: Used sudo apt install scala

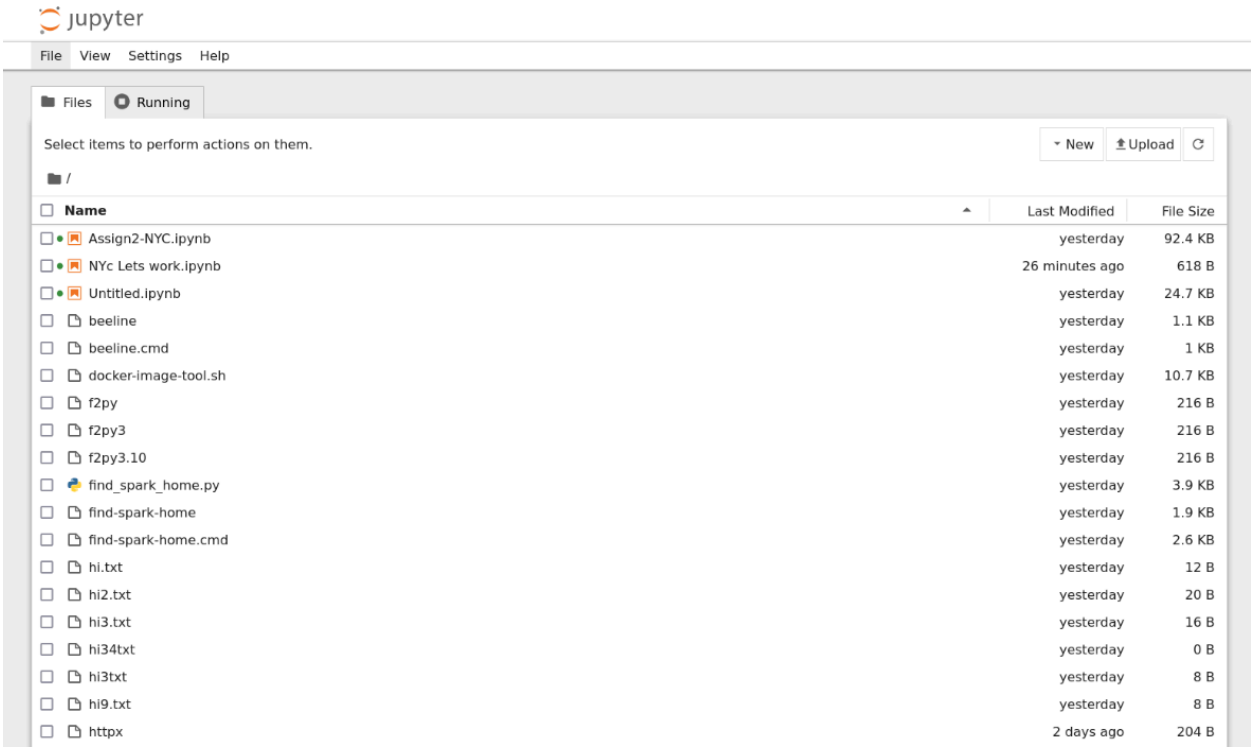
```
exouser@assign2vp:~$ sudo apt install scala
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libhawtjni-runtime-java libjansi-java libjansi-native-java libjline2-java
  scala-library scala-parser-combinators scala-xml
Suggested packages:
  scala-doc
The following NEW packages will be installed:
  libhawtjni-runtime-java libjansi-java libjansi-native-java libjline2-java
  scala scala-library scala-parser-combinators scala-xml
0 upgraded, 8 newly installed, 0 to remove and 0 not upgraded.
Need to get 25.1 MB of archives.
After this operation, 28.6 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 libhawtjni-runtime-java all 1.1
7-1 [28.8 kB]
Get:2 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 libjansi-native-java all 1.8-1
[23.8 kB]
Get:3 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 libjansi-java all 1.18-1 [56.8
kB]
Get:4 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 libjline2-java all 2.14.6-4 [15
0 kB]
Get:5 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 scala-library all 2.11.12-5 [95
86 kB]
Get:6 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 scala-parser-combinators all 1.
0.3-3.1 [365 kB]
Get:7 http://nova.clouds.archive.ubuntu.com/ubuntu jammy/universe amd64 scala-xml all 1.0.3-3.1 [615 kB
```

Step 8: Started the Jupyter

```
exouser@assign2vp:~/.local/bin$ jupyter-notebook --no-browser
[I 2024-04-13 03:34:23.523 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2024-04-13 03:34:23.527 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2024-04-13 03:34:23.531 ServerApp] jupyterlab | extension was successfully linked.
[I 2024-04-13 03:34:23.535 ServerApp] notebook | extension was successfully linked.
[I 2024-04-13 03:34:23.775 ServerApp] notebook_shim | extension was successfully linked.
[I 2024-04-13 03:34:23.789 ServerApp] notebook_shim | extension was successfully loaded.
[I 2024-04-13 03:34:23.791 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2024-04-13 03:34:23.792 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2024-04-13 03:34:23.793 LabApp] JupyterLab extension loaded from /home/exouser/.local/lib/python3.10/site-packages/jupyterla
b
[I 2024-04-13 03:34:23.793 LabApp] JupyterLab application directory is /home/exouser/.local/share/jupyter/lab
[I 2024-04-13 03:34:23.793 LabApp] Extension Manager is 'pypi'.
[I 2024-04-13 03:34:23.827 ServerApp] jupyterlab | extension was successfully loaded.
[I 2024-04-13 03:34:23.830 ServerApp] notebook | extension was successfully loaded.
[I 2024-04-13 03:34:23.830 ServerApp] Serving notebooks from local directory: /home/exouser/.local/bin
[I 2024-04-13 03:34:23.830 ServerApp] Jupyter Server 2.13.0 is running at:
[I 2024-04-13 03:34:23.830 ServerApp] http://localhost:8888/tree?token=d4d0b5a1c6475af5182c5a38071612e59ce803187e88107d
[I 2024-04-13 03:34:23.830 ServerApp]     http://127.0.0.1:8888/tree?token=d4d0b5a1c6475af5182c5a38071612e59ce803187e88107d
[I 2024-04-13 03:34:23.830 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2024-04-13 03:34:23.834 ServerApp]

    To access the server, open this file in a browser:
        file:///home/exouser/.local/share/jupyter/runtime/jpserver-342287-open.html
    Or copy and paste one of these URLs:
        http://localhost:8888/tree?token=d4d0b5a1c6475af5182c5a38071612e59ce803187e88107d
        http://127.0.0.1:8888/tree?token=d4d0b5a1c6475af5182c5a38071612e59ce803187e88107d
```

Step 9: Can access Jupyter Notebook



PART 1 NYC Parking Violation Dataset

Spark Session:

VAISHNAVI VISHWAS PAWAR

ENGINEERING CLOUD COMPUTING - ASSIGNMENT 2

PART 1 NYC Dataset

[1]:
```
# Importing all the necessary Python libraries & packages
import os
import numpy as np
import pandas as pd
import shutil
import pyspark
```

[2]:
```
#Importing functions and types from PySpark SQL to manipulate data.
from pyspark.sql import SparkSession, SQLContext, Window, types

#Importng specific functions that are widely used in data processing.
import pyspark.sql.functions as pyspkfnt
from pyspark.sql.functions import when, count, col, sum, regexp_replace

# Machine Learning & feature engineering tools from PySpark MLlib
from pyspark.ml import clustering as ml_clustering, feature as ml_feature

# To manage Spark functionality
from pyspark import SparkContext
IntegerType = types.IntegerType
VectorAssembler = ml_feature.VectorAssembler
KMeans = ml_clustering.KMeans

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
```

Successful Spark Connection:

```
[7]: nycSpark
```

```
[7]: SparkSession - in-memory

SparkContext

Spark UI

Version    v3.1.1
Master     local[*]
AppName    VaishnaviP_Assignment2_NYC
```

<u>Data File Used is</u>: Parking_Violations_Issued_-_Fiscal_Year_2024_20240410.csv

Loading of the NYC Dataset & Displaying Rows and Columns:

```
[8]: #Define the path to the dataset for NYC Parking Violations for Fiscal Year 2024
DatasetPath = '/home/exouser/Downloads/Parking_Violations_Issued_-_Fiscal_Year_2024_20240410.csv'

#Loading the dataset as a DataFrame named ParkingDataNYC
ParkingDataNYC = nycSpark.read.format("csv").option("header", "true").option("inferSchema", "true").load(DatasetPath)
```

```
[9]: #Total Rows & Columns of the Dataset
print("Total Rows in the Dataset of NYC: ", ParkingDataNYC.count())
print("Total Columns in the Dataset of NYC: ", len(ParkingDataNYC.columns))
```

```
[Stage 2:==================================================>       (13 + 2) / 15]
Total Rows in the Dataset of NYC:  10717482
Total Columns in the Dataset of NYC:  43
```

Displaying two records from the Dataset:

```
[11]: #Displaying 2 Records from the dataset
ParkingDataNYC.show(n=2, truncate=False, vertical=True)
```

```
-RECORD 0-----------------------------------------
 Summons Number                  | 1159637337
 Plate ID                        | KZH2758
 Registration State              | NY
 Plate Type                      | PAS
 Issue Date                      | 06/09/2023
 Violation Code                  | 67
 Vehicle Body Type               | VAN
 Vehicle Make                    | HONDA
 Issuing Agency                  | P
 Street Code1                    | 0
 Street Code2                    | 0
 Street Code3                    | 0
 Vehicle Expiration Date         | 20250201
 Violation Location              | 43
 Violation Precinct              | 43
 Issuer Precinct                 | 43
 Issuer Code                     | 972773
 Issuer Command                  | 0043
 Issuer Squad                    | 0000
 Violation Time                  | 0911A
 Time First Observed             | null
 Violation County                | BX
 Violation In Front Of Or Opposite | null
 House Number                    | null
 Street Name                     | I/O TAYLOR AVE
 Intersecting Street             | GUERLAIN
 Date First Observed             | 0
 Law Section                     | 408
 Sub Division                    | E5
 Violation Legal Code            | null
 Days Parking In Effect          | BBBBBBB
 From Hours In Effect            | ALL
 To Hours In Effect              | ALL
 Vehicle Color                   | BLUE
 Unregistered Vehicle?           | 0
```

Database Schema:

```
[8]: #Reviewing Schema of the Dataset
     ParkingDataNYC.printSchema()
```

```
root
 |-- Summons Number: long (nullable = true)
 |-- Plate ID: string (nullable = true)
 |-- Registration State: string (nullable = true)
 |-- Plate Type: string (nullable = true)
 |-- Issue Date: string (nullable = true)
 |-- Violation Code: integer (nullable = true)
 |-- Vehicle Body Type: string (nullable = true)
 |-- Vehicle Make: string (nullable = true)
 |-- Issuing Agency: string (nullable = true)
 |-- Street Code1: integer (nullable = true)
 |-- Street Code2: integer (nullable = true)
 |-- Street Code3: integer (nullable = true)
 |-- Vehicle Expiration Date: integer (nullable = true)
 |-- Violation Location: integer (nullable = true)
 |-- Violation Precinct: integer (nullable = true)
 |-- Issuer Precinct: integer (nullable = true)
 |-- Issuer Code: integer (nullable = true)
 |-- Issuer Command: string (nullable = true)
 |-- Issuer Squad: string (nullable = true)
 |-- Violation Time: string (nullable = true)
 |-- Time First Observed: string (nullable = true)
 |-- Violation County: string (nullable = true)
 |-- Violation In Front Of Or Opposite: string (nullable = true)
 |-- House Number: string (nullable = true)
 |-- Street Name: string (nullable = true)
 |-- Intersecting Street: string (nullable = true)
 |-- Date First Observed: integer (nullable = true)
 |-- Law Section: integer (nullable = true)
 |-- Sub Division: string (nullable = true)
 |-- Violation Legal Code: string (nullable = true)
 |-- Days Parking In Effect    : string (nullable = true)
 |-- From Hours In Effect: string (nullable = true)
 |-- To Hours In Effect: string (nullable = true)
 |-- Vehicle Color: string (nullable = true)
 |-- Unregistered Vehicle?: integer (nullable = true)
 |-- Vehicle Year: integer (nullable = true)
 |-- Meter Number: string (nullable = true)
 |-- Feet From Curb: integer (nullable = true)
```

Displaying the top Record of the dataset

[13]:
```
#Displaying only the top record
ParkingDataNYC.selectExpr("*").show(1, vertical=True)
```

```
-RECORD 0-----------------------------------------
 Summons Number                  | 1159637337
 Plate ID                        | KZH2758
 Registration State              | NY
 Plate Type                      | PAS
 Issue Date                      | 06/09/2023
 Violation Code                  | 67
 Vehicle Body Type               | VAN
 Vehicle Make                    | HONDA
 Issuing Agency                  | P
 Street Code1                    | 0
 Street Code2                    | 0
 Street Code3                    | 0
 Vehicle Expiration Date         | 20250201
 Violation Location              | 43
 Violation Precinct              | 43
 Issuer Precinct                 | 43
 Issuer Code                     | 972773
 Issuer Command                  | 0043
 Issuer Squad                    | 0000
 Violation Time                  | 0911A
 Time First Observed             | null
 Violation County                | BX
 Violation In Front Of Or Opposite | null
 House Number                    | null
 Street Name                     | I/O TAYLOR AVE
 Intersecting Street             | GUERLAIN
 Date First Observed             | 0
 Law Section                     | 408
 Sub Division                    | E5
 Violation Legal Code            | null
 Days Parking In Effect          | BBBBBBB
 From Hours In Effect            | ALL
 To Hours In Effect              | ALL
 Vehicle Color                   | BLUE
 Unregistered Vehicle?           | 0
 Vehicle Year                    | 2006
 Meter Number                    | -
 Feet From Curb                  | 0
```

Null Values detected in the Dataset:

```
-RECORD 0-----------------------------------
 Summons Number                      | 0
 Plate ID                            | 1
 Registration State                  | 0
 Plate Type                          | 0
 Issue Date                          | 0
 Violation Code                      | 0
 Vehicle Body Type                   | 28486
 Vehicle Make                        | 10679
 Issuing Agency                      | 0
 Street Code1                        | 0
 Street Code2                        | 0
 Street Code3                        | 0
 Vehicle Expiration Date             | 0
 Violation Location                  | 4923863
 Violation Precinct                  | 0
 Issuer Precinct                     | 0
 Issuer Code                         | 0
 Issuer Command                      | 4918591
 Issuer Squad                        | 5292644
 Violation Time                      | 336
 Time First Observed                 | 10087137
 Violation County                    | 102892
 Violation In Front Of Or Opposite   | 4973482
 House Number                        | 5015875
 Street Name                         | 1507
 Intersecting Street                 | 4980009
 Date First Observed                 | 0
 Law Section                         | 0
 Sub Division                        | 1767
 Violation Legal Code                | 5799044
 Days Parking In Effect              | 5014718
 From Hours In Effect                | 7338955
 To Hours In Effect                  | 7338965
 Vehicle Color                       | 1015121
 Unregistered Vehicle?               | 10490502
 Vehicle Year                        | 0
 Meter Number                        | 9381186
 Feet From Curb                      | 0
 Violation Post Code                 | 5519326
 Violation Description               | 227812
 No Standing or Stopping Violation   | 10717482
 Hydrant Violation                   | 10717482
```

Pre-processing Steps and Handling Null Values

Performing Pre-Processing steps & Handling Null Values

#Dropping rows with missing values in the specific columns that are required for the Assignment in the further steps

```
16]: ParkingDataNYC = ParkingDataNYC.dropna(subset=['Violation Location'])

17]: ParkingDataNYC = ParkingDataNYC.dropna(subset=['Violation Time'])

18]: ParkingDataNYC = ParkingDataNYC.dropna(subset=['Vehicle Color'])

19]: ParkingDataNYC = ParkingDataNYC.dropna(subset=['Vehicle Body Type'])

20]: #To verify whether missing values are removed from the designated columns
     from pyspark.sql.functions import isnan, when, count, col

     #Displaying columns with their null counts
     NullCnts = ParkingDataNYC.select([count(when(col(c).isNull(), c)).alias(c) for c in ParkingDataNYC.columns])
     NullCnts.show(vertical=True)

     [Stage 9:=====================================================>   (14 + 1) / 15]
     -RECORD 0-----------------------------------
      Summons Number                  | 0
      Plate ID                        | 0
      Registration State              | 0
      Plate Type                      | 0
      Issue Date                      | 0
      Violation Code                  | 0
      Vehicle Body Type               | 0
      Vehicle Make                    | 5004
      Issuing Agency                  | 0
      Street Code1                    | 0
      Street Code2                    | 0
      Street Code3                    | 0
      Vehicle Expiration Date         | 0
      Violation Location              | 0
      Violation Precinct              | 0
```

Verifying whether the Null Values in the Dataset are deleted :

Null values detected in the NYC dataset.

```
[14]: from pyspark.sql.functions import isnan, when, count, col

      #Displaying columns with their null counts
      NullCnts = ParkingDataNYC.select([count(when(col(c).isNull(), c)).alias(c) for c in ParkingDataNYC.columns])
      NullCnts.show(vertical=True)

      [Stage 7:=============================================>           (12 + 3) / 15]
      -RECORD 0-----------------------------------
       Summons Number                  | 0
       Plate ID                        | 1
       Registration State              | 0
       Plate Type                      | 0
       Issue Date                      | 0
       Violation Code                  | 0
       Vehicle Body Type               | 28486
       Vehicle Make                    | 10679
       Issuing Agency                  | 0
       Street Code1                    | 0
       Street Code2                    | 0
       Street Code3                    | 0
       Vehicle Expiration Date         | 0
       Violation Location              | 4923863
       Violation Precinct              | 0
       Issuer Precinct                 | 0
       Issuer Code                     | 0
       Issuer Command                  | 4918591
       Issuer Squad                    | 5292644
       Violation Time                  | 336
       Time First Observed             | 10087137
       Violation County                | 102892
       Violation In Front Of Or Opposite | 4973482
       House Number                    | 5015875
       Street Name                     | 1507
       Intersecting Street             | 4980009
```

Q. 1] When are tickets most likely to be issued?

Query :

```
nycSpark.sql("""
   SELECT `Violation Time`,
          COUNT(*) AS Ticket_Frequency
   FROM DatasetViewNYC
   GROUP BY `Violation Time`
   ORDER BY Ticket_Frequency DESC
""")
```

Q. 1] When are tickets most likely to be issued?

[22]:
```
# Running a SQL query to calculate ticket frequency based on violation time
Ans1= nycSpark.sql("""
    SELECT `Violation Time`,
           COUNT(*) AS Ticket_Frequency
    FROM DatasetViewNYC
    GROUP BY `Violation Time`
    ORDER BY Ticket_Frequency DESC
""")

#Display the query result
Ans1.show()
```

```
[Stage 11:=============================================>           (12 + 3) / 15]
+--------------+----------------+
|Violation Time|Ticket_Frequency|
+--------------+----------------+
|         0836A|           16176|
|         0838A|           15559|
|         0839A|           15544|
|         0840A|           15381|
|         0906A|           15142|
|         0841A|           15017|
|         0837A|           14901|
|         0842A|           14724|
|         0908A|           14488|
|         0843A|           14430|
|         0845A|           14408|
|         0910A|           14388|
|         0909A|           14379|
|         0907A|           14263|
|         0844A|           14226|
|         1140A|           13978|
|         0846A|           13899|
|         1141A|           13789|
|         1139A|           13713|
|         0911A|           13706|
+--------------+----------------+
only showing top 20 rows
```

Answer 1: According to the question, we need to determine when the tickets were most likely issued. As a result, I performed an SQL query to sort the ticket frequency in descending order, giving us the tickets that were most frequently issued while also displaying the violation time. The above result indicates that there were a significant number of violations registered in the dataset around **8:36 AM (0836A)**. A total of **16,176** represents the **frequency of violations** observed at this time.

Answer 1: According to the question, we need to determine when the tickets were most likely issued. As a result, I performed an SQL query to sort the ticket frequency in descending order, giving us the tickets that were most frequently issued while also displaying the violation time. The above result indicates that there were a significant number of violations registered in the dataset around 8:36 AM (0836A). A total of 16,176 represents the frequency of violations observed at this time.

Q. 2] What are the most common years and types of cars to be ticketed?

Query:

```
nycSpark.sql("""
   SELECT `Issue Year`,
        `Vehicle Body Type` as `Car Type`,
        COUNT(*) AS `Violation_Count`
   FROM DatasetViewNYC
   WHERE (`Vehicle Year` > 0)
   GROUP BY `Vehicle Body Type`, `Issue Year`
   ORDER BY `Violation_Count` DESC
""")
```

Answer 2: According to the question, we must determine the most common years and types of cars to be ticketed. As a result, I performed a SQL query to determine the type of cars to be ticketed based on the most common year. The above result discloses that the Most Common Year for car tickets is 2023.
The most often ticketed Type of Car is identified as "SUBN", with the Violation Count totaling 1455740.

Q. 2] What are the most common years and types of cars to be ticketed?

[23]:
```
# Executing a SQL query to count violations by car body type and issue year
Ans2= nycSpark.sql("""
    SELECT `Issue Year`,
           `Vehicle Body Type` as `Car Type`,
           COUNT(*) AS `Violation_Count`
    FROM DatasetViewNYC
    WHERE (`Vehicle Year` > 0)
    GROUP BY `Vehicle Body Type`, `Issue Year`
    ORDER BY `Violation_Count` DESC
""")

Ans2.show(15)
```

```
[Stage 13:=================================================>   (14 + 1) / 15]
+----------+--------+---------------+
|Issue Year|Car Type|Violation_Count|
+----------+--------+---------------+
|      2023|    SUBN|        1455740|
|      2023|    4DSD|         762700|
|      2024|    SUBN|         470615|
|      2023|     VAN|         438855|
|      2024|    4DSD|         233471|
|      2024|     VAN|         137466|
|      2023|    PICK|         100055|
|      2023|    DELV|          85359|
|      2023|    2DSD|          56158|
|      2023|     SDN|          49267|
|      2023|    REFG|          39034|
|      2024|    PICK|          32876|
|      2024|    DELV|          25891|
|      2024|    2DSD|          16746|
|      2023|    CONV|          15146|
+----------+--------+---------------+
only showing top 15 rows
```

Answer 2: According to the question, we must determine the most common years and types of cars to be ticketed. As a result, I performed a SQL query to determine the type of cars to be ticketed based on the most common year. The above result discloses that the **Most Common Year** for car tickets is **2023**.

The most often ticketed **Type of Car** is identified as **"SUBN"**, with the **Violation Count** totaling **1455740**.

Q. 3]Where are tickets most commonly issued?

Query:

```
nycSpark.sql("""
   SELECT `Violation Location`,
          COUNT(*) AS `Total_Tickets`
   FROM DatasetViewNYC
   GROUP BY `Violation Location`
   ORDER BY `Total_Tickets` DESC
""")
```

Approach : To determine where the tickets were issued I have used Violation Location in the query.

Answer 3: According to the question, we must determine where tickets are typically issued. Thus, I executed a SQL query to identify the location with the most tickets.
The below results discloses, that the Location for the majority of tickets is 19.

Q. 3] Where are tickets most commonly issued?

```
24]: # Executing a SQL query to determine the location of the tickets with the most violations

Ans3 = nycSpark.sql("""
    SELECT `Violation Location`,
           COUNT(*) AS `Total_Tickets`
    FROM DatasetViewNYC
    GROUP BY `Violation Location`
    ORDER BY `Total_Tickets` DESC
""")

Ans3.show(12)
```

```
[Stage 15:=====================================================>   (14 + 1) / 15]
+------------------+-------------+
|Violation Location|Total_Tickets|
+------------------+-------------+
|                19|       275694|
|               114|       211702|
|                 6|       207346|
|                13|       188822|
|                14|       177747|
|               109|       153290|
|                 1|       147416|
|                18|       147053|
|                 9|       141548|
|               115|       135019|
|                61|       116063|
|                66|       115531|
+------------------+-------------+
only showing top 12 rows
```

Answer 3: According to the question, we must determine where tickets are typically issued. Thus, I executed a SQL query to identify the location with the most tickets.

Above results discloses, the **Location** for the majority of tickets is **19**.

Q 4]Which color of the vehicle is most likely to get a ticket?

Query:

```
nycSpark.sql("""
   SELECT `Vehicle Color`,
          COUNT(*) AS CntOfTicket
   FROM DatasetViewNYC
   GROUP BY `Vehicle Color`
   ORDER BY CntOfTicket DESC
""")
```

Answer 4: Based on the question, we must decide which color of the vehicle is most likely to receive a ticket. As a result, I ran a SQL query to get the most ticketed vehicle color. The above result reveals that the Vehicle Color for the majority of tickets is WH. Ticket count for this vehicle color: 1100571.

As an outcome, it is most likely that a White (WH) car will receive a ticket.

Q. 4] Which color of the vehicle is most likely to get a ticket?

```
[25]: # Executing a SQL query to determine the most ticketed vehicle color

Ans4 = nycSpark.sql("""
    SELECT `Vehicle Color`,
           COUNT(*) AS CntOfTicket
    FROM DatasetViewNYC
    GROUP BY `Vehicle Color`
    ORDER BY CntOfTicket DESC
""")
Ans4.show(8)
```

```
[Stage 17:==================================================>       (13 + 2) / 15]
+-------------+-----------+
|Vehicle Color|CntOfTicket|
+-------------+-----------+
|           WH|    1100571|
|           GY|    1013560|
|           BK|     867338|
|        WHITE|     598915|
|        BLACK|     389161|
|           BL|     348480|
|         GREY|     301436|
|           RD|     194491|
+-------------+-----------+
only showing top 8 rows
```

Answer 4: Based on the question, we must decide which color of the vehicle is most likely to receive a ticket. As a result, I ran a SQL query to get the most ticketed vehicle color. The above result reveals that the **Vehicle Color** for the majority of tickets is **WH**. **Ticket count** for this vehicle color: **1100571**.

As an outcome, it is most likely that a **White (WH)** car will receive a ticket.

Based on a K-Means algorithm, please try to answer the following question:

•Q. 5] Given a Black vehicle parking illegally at 34510, 10030, 34050 (street codes) the probability that it will get an ticket? (very rough prediction).

Successful Spark Connection:

Based on a K-Means algorithm,

Q. 5] Given a Black vehicle parking illegally at 34510, 10030, 34050 (street codes). What is the probability that it will get an ticket?

```
]: # Initialize SparkSession for the NYC parking data analysis project using K-means clustering
nycSpark = SparkSession.builder \
    .appName("VaishnaviP_Assignment2_NYC-Kmeans") \
    .master("local[*]") \
    .config("spark.driver.memory", "32G") \
    .config("spark.ui.showConsoleProgress", True) \
    .config("spark.sql.execution.arrow.pyspark.enabled", True) \
    .config("spark.sql.session.timeZone", "UTC") \
    .config("spark.sql.repl.eagerEval.enabled", True) \
    .getOrCreate()
```

```
]: nycSpark
```

```
]: SparkSession - in-memory

SparkContext

Spark UI

Version      v3.1.1
Master       local[*]
AppName      VaishnaviP_Assignment2_NYC-Kmeans
```

Approach:

1. Generate Feature Vector: I have created a new column called "Street_codes," which is a vectorized version of the "Street Code1," "Street Code2," and "Street Code3" columns created with the Spark VectorAssembler.

2. ExecuteKmeans: This function takes a Spark DataFrame inputData with a "Street_codes" column as input, initializes and fits a K-Means clustering model with k=4 clusters, and returns the converted DataFrame with the cluster centers as a NumPy array.

```
#This_function uses the supplied street code columns from the Dataframe to create a feature_Vector.
def GenerateFeatureVector(InputNycData):
    #Create a feature vector using the DataFrame's given street code columns.
    vectorCreator = VectorAssembler(inputCols=["Street Code1", "Street Code2", "Street Code3"], outputCol="AggregatedCodes")
    return vectorCreator.transform(InputNycData)

def executeKmeans(InputNycData):
    #Fit K-Means clustering with a predetermined number of clusters to the data set.
    kmeansCluster = KMeans(k=4, featuresCol="AggregatedCodes")
    Clustermodel = kmeansCluster.fit(InputNycData.select('AggregatedCodes'))
    clusterCenters = np.array(Clustermodel.clusterCenters()).astype(float)
    return Clustermodel.transform(InputNycData).cache(), clusterCenters
```

3. This function uses a Spark DataFrame with a 'prediction' and a 'VehicleColor' column to determine the likelihood of having black-colored vehicles in each cluster. The blackColor list is used to specify the black colors to use.

```
#Function to determine the amount of black cars in the grouped data by using Prediction_clusters.
def estimateBlackpresence(InputNycData, blackColorArray):
    #Group data by 'prediction' and determine the number of black and total vehicles.
    blackVehicleRatio = InputNycData.groupBy('prediction').agg(
        pyspkfnt.sum(pyspkfnt.when(pyspkfnt.col('Vehicle Color').isin(blackColorArray), 1)).alias('BlackCnt'),
        pyspkfnt.count('Vehicle Color').alias('NumVehicles')
    ).orderBy('prediction')

    #Calculates the Probability of Car_being_Black
    return blackVehicleRatio.select(
        'prediction',
        'BlackCnt',
        'NumVehicles',
        (pyspkfnt.col('BlackCnt') /pyspkfnt.col('NumVehicles')).alias('Probability')
    )
```

4. This function calculates the Euclidean distance between each cluster center and a collection of streets and returns the nearest cluster's index.

```
[35]: #This Function locates the cluster whose center is closest to the specified streets.
def computeClosestClusterIdx(CoordinatesOfStreet, clusterCenters):
    kmeansCenterID = 0
    bestNearestDist = float("inf")

    idx= 0
    while idx < len(clusterCenters):
        #Compute the Euclidean_distance between the street_data & the cluster_center.
        distToCluster = np.sum(np.power((np.array(CoordinatesOfStreet) - clusterCenters[idx]), 2))

        #Update the nearest_distance and clusterID if a closest cluster is found.
        if distToCluster < bestNearestDist:
            bestNearestDist = distToCluster
            kmeansCenterID = idx

        idx +=1

    return kmeansCenterID
```

5. Printing the cluster ID and probability of that specific cluster using the following function.
displayProbabilityForCluster and computeAndPrintProb are the primary functions that call all other functions.

```
36]: #Function_to_Print the probability for the Particular_cluster
def displayProbabilityForCluster(kmeansCenterID, blackPrediction):
    print('Given Street Code (34510, 10030, 34050) has Cluster ID:', kmeansCenterID)
    print("-------------------------------------------------------------------------")
    print("Probability in that Particular Cluster:")
    blackPrediction.filter(pyspkfnt.col('prediction') == kmeansCenterID).show()
```

```
37]: #Function to_calculate and print_probability.
def computeAndPrintProb(InputNycData, blackColorArray, streetAttributes):
    #In the input_data of NYC_Dataset, vectorize the street codes.
    InputNycData = GenerateFeatureVector(InputNycData)

    # Initialize K-Means_clustering and get centers_Of_Clusters
    Clustermodel, clusterCenters = executeKmeans(InputNycData)

    #Calculate_probability of black_Color
    blackPrediction = estimateBlackpresence(Clustermodel, blackColorArray)

    #Locate the cluster nearest the street_code.
    kmeansCenterID = computeClosestClusterIdx(streetAttributes, clusterCenters)

    #Print the probability for the Particular_cluster
    displayProbabilityForCluster(kmeansCenterID, blackPrediction)
```

6. The following are the possible black hues for automobiles and street codes for illegal parking.

```
: %%capture cap --no-stdout

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

blackColorArray=['BLK.', 'Black', 'BC', 'BLAC', 'BK/', 'BLK', 'BK.', 'BCK', 'BK', 'B LAC']
streetAttributes=[34510, 10030, 34050]
computeAndPrintProb(ParkingDataNYC,blackColorArray,streetAttributes)
```

7. The function call and the output of Kmeans are shown below.

```
streetAttributes=[34510, 10030, 34050]
computeAndPrintProb(ParkingDataNYC,blackColorArray,streetAttributes)
```

Output:

```
Given Street Code (34510, 10030, 34050) has Cluster ID: 2
--------------------------------------------------------------------------
Probability in that Particular Cluster:
+----------+--------+-----------+-------------------+
|prediction|BlackCnt|NumVehicles|        Probability|
+----------+--------+-----------+-------------------+
|         2|  226416|    1517941|0.14915994758689566|
+----------+--------+-----------+-------------------+
```

Answer: For K value 4, I created the model and the probability is **0.14915994758689566.**

PART 2 NBA SHOT LOGS:

Data File: shot_logs.csv

Spark Session:

```
[4]: #Initialized the Spark session and enabled its evaluation for result
     nycSpark = pyspark.sql.SparkSession.builder.appName("VaishnaviP_Assignment2_NBA").getOrCreate()
     nycSpark.conf.set("spark.sql.repl.eagerEval.enabled", True)

[5]: nycSpark

[5]: SparkSession - in-memory

     SparkContext

     Spark UI

     Version    v3.1.1
     Master     local[*]
     AppName    VaishnaviP_Assignment2_NBA
```

Loading the NBA Dataset:

```
#Define the path to the dataset for NYC Parking Violations for Fiscal Year 2024
DatasetPath = '/home/exouser/Downloads/shot_logs.csv'

#Loading the dataset as a DataFrame named ParkingDataNYC
NBADataset = nycSpark.read.format("csv").option("header", "true").option("inferSchema", "true").load(DatasetPath)
```

```
#Total Rows & Columns of the Dataset
print("Total Rows in the Dataset of NBA: ", NBADataset.count())
print("Total Columns in the Dataset of NBA: ", len(NBADataset.columns))
```

```
Total Rows in the Dataset of NBA:  128069
Total Columns in the Dataset of NBA:  21
```

Displaying two records of the Dataset:

```
[8]: #Displaying 2 Records from the dataset
     NBADataset.show(n=2,vertical=True)
```

```
-RECORD 0-----------------------------------------
 GAME_ID                    | 21400899
 MATCHUP                    | MAR 04, 2015 - CH...
 LOCATION                   | A
 W                          | W
 FINAL_MARGIN               | 24
 SHOT_NUMBER                | 1
 PERIOD                     | 1
 GAME_CLOCK                 | 1:09
 SHOT_CLOCK                 | 10.8
 DRIBBLES                   | 2
 TOUCH_TIME                 | 1.9
 SHOT_DIST                  | 7.7
 PTS_TYPE                   | 2
 SHOT_RESULT                | made
 CLOSEST_DEFENDER           | Anderson, Alan
 CLOSEST_DEFENDER_PLAYER_ID | 101187
 CLOSE_DEF_DIST             | 1.3
 FGM                        | 1
 PTS                        | 2
 player_name                | brian roberts
 player_id                  | 203148
-RECORD 1-----------------------------------------
 GAME_ID                    | 21400899
 MATCHUP                    | MAR 04, 2015 - CH...
 LOCATION                   | A
 W                          | W
 FINAL_MARGIN               | 24
 SHOT_NUMBER                | 2
 PERIOD                     | 1
 GAME_CLOCK                 | 0:14
 SHOT_CLOCK                 | 3.4
 DRIBBLES                   | 0
 TOUCH_TIME                 | 0.8
 SHOT_DIST                  | 28.2
 PTS_TYPE                   | 3
 SHOT_RESULT                | missed
 CLOSEST_DEFENDER           | Bogdanovic, Bojan
 CLOSEST_DEFENDER_PLAYER_ID | 202711
 CLOSE_DEF_DIST             | 6.1
```

Data Schema:

```
[9]: #Reviewing Schema of the Dataset
     NBADataset.printSchema()

root
 |-- GAME_ID: integer (nullable = true)
 |-- MATCHUP: string (nullable = true)
 |-- LOCATION: string (nullable = true)
 |-- W: string (nullable = true)
 |-- FINAL_MARGIN: integer (nullable = true)
 |-- SHOT_NUMBER: integer (nullable = true)
 |-- PERIOD: integer (nullable = true)
 |-- GAME_CLOCK: string (nullable = true)
 |-- SHOT_CLOCK: double (nullable = true)
 |-- DRIBBLES: integer (nullable = true)
 |-- TOUCH_TIME: double (nullable = true)
 |-- SHOT_DIST: double (nullable = true)
 |-- PTS_TYPE: integer (nullable = true)
 |-- SHOT_RESULT: string (nullable = true)
 |-- CLOSEST_DEFENDER: string (nullable = true)
 |-- CLOSEST_DEFENDER_PLAYER_ID: integer (nullable = true)
 |-- CLOSE_DEF_DIST: double (nullable = true)
 |-- FGM: integer (nullable = true)
 |-- PTS: integer (nullable = true)
 |-- player_name: string (nullable = true)
 |-- player_id: integer (nullable = true)
```

<u>Question 1:</u> For each pair of the players (A, B), we define the fear sore of A when facing B is the hit rate, such that B is closet defender when A is shooting. Based on the fear sore, for each player, please find out who is his ”most unwanted defender”.

<u>Approach used:</u>

1. Two conditions, shotScored and shotMissed, are used to indicate whether a basketball shot was made or missed. The NBA information is grouped by player and closest defender, and a new DataFrame, gameShotResults, is generated to aggregate the counts of made and missed shots for each pair. This provides insights into scoring performance.

```
[7]: #Function Calculates if the shot was scored or not
shotScored= pyspkfnt.when(pyspkfnt.col("SHOT_RESULT") == "made", 1).otherwise(0)
shotMissed = pyspkfnt.when(pyspkfnt.col("SHOT_RESULT") == "missed", 1).otherwise(0)

#Data is aggregated by player and closest defender.
gameShotResults = NBADataset.groupBy(
    pyspkfnt.col("player_id").alias("ID of Player"),
    pyspkfnt.col("CLOSEST_DEFENDER_PLAYER_ID").alias("ID of Defender")
).agg(
    pyspkfnt.sum(shotScored).alias("Player Scored"),
    pyspkfnt.sum(shotMissed).alias("Player Did Not Scored")
)

gameShotResults.show(12)

+-----------+-------------+-------------+--------------------+
|ID of Player|ID of Defender|Player Scored|Player Did Not Scored|
+-----------+-------------+-------------+--------------------+
|     203148|       101179|            0|                   1|
|     202687|       201980|            1|                   0|
|       2744|         1717|            0|                   2|
|     203469|       202329|            1|                   1|
|     201945|       202322|            0|                   3|
|     202689|       202699|            6|                   8|
|     202689|       203924|            1|                   0|
|     203077|         2730|            1|                   0|
|     203077|       201584|            2|                   0|
|     202362|       201188|            2|                   0|
|     202330|       201978|            2|                   1|
|     202324|       203135|            1|                   2|
+-----------+-------------+-------------+--------------------+
only showing top 12 rows
```

2. A new column "HitRate" has been introduced to the DataFrame gameShotResults, reflecting the percentage of made shots to total shots (made + missed). This provides a measure of scoring effectiveness for each player and defensive pair.

```
#calculate shooting_accuracy based on successful and missed shots.
shotsAccuracy = gameShotResults.withColumn(
    "HitRate",
    pyspkfnt.col("Player Scored") / (pyspkfnt.col("Player Scored") + pyspkfnt.col("Player Did Not Scored"))
)
shotsAccuracy.show(8)
```

```
+------------+--------------+-------------+--------------------+------------------+
|ID of Player|ID of Defender|Player Scored|Player Did Not Scored|           HitRate|
+------------+--------------+-------------+--------------------+------------------+
|      203148|        101179|            0|                   1|               0.0|
|      202687|        201980|            1|                   0|               1.0|
|        2744|          1717|            0|                   2|               0.0|
|      203469|        202329|            1|                   1|               0.5|
|      201945|        202322|            0|                   3|               0.0|
|      202689|        202699|            6|                   8|0.42857142857142855|
|      202689|        203924|            1|                   0|               1.0|
|      203077|          2730|            1|                   0|               1.0|
+------------+--------------+-------------+--------------------+------------------+
only showing top 8 rows
```

Pre- Processing and Handling Null Values:

1. The first stage is filtering out rows in the DataFrame gameShotResults with a non-null "HitRate" column, removing duplicate rows based on "ID of Player" and "HitRate," calculating the minimal "HitRate" for each player, and joining this information back to the original DataFrame.

2. The NBA dataset is used to construct a new DataFrame called gameShotResults, which contains information on each player's most undesirable defender. The process removes duplicate entries based on "ID of Player" and "ID of Defender" and presents the first 10 rows with "Name of the Player" and "Most Unwanted Defender by the Player" columns.

Pre-Processing & Handling Null Values

```
#Remove any rows (null values) from the DataFrame where the Shooting Accuracy is not calculated.
shotsAccuracy = shotsAccuracy.filter(pyspkfnt.col("HitRate").isNotNull())
```

```
#Drop Duplicates_in_shotsAccuracy
shotsAccuracy = shotsAccuracy.dropDuplicates(subset=["ID of Player", "HitRate"])
```

```
#Calculate each player's minimal shooting_accuracy by grouping the DataFrame by ID_of_Player
PlayerPerformance = shotsAccuracy.groupBy("ID of Player").agg(pyspkfnt.min("HitRate").alias("HitRate"))
```

```
shotsAccuracy = shotsAccuracy.join(
    PlayerPerformance,
    ["ID of Player", "HitRate"]
).select("ID of Player", "ID of Defender")

shotsAccuracy = shotsAccuracy.join(
    NBADataset,
    (NBADataset["player_id"] == shotsAccuracy["ID of Player"]) & (NBADataset["CLOSEST_DEFENDER_PLAYER_ID"] == shotsAccuracy["ID of Defender"])
).withColumn("Name of the Player", col("player_name")).withColumn("Most Unwanted Defender by the Player", col("CLOSEST_DEFENDER"))

shotsAccuracy = shotsAccuracy.dropDuplicates(["ID of Player", "ID of Defender"])

shotsAccuracy.select("Name of the Player", "Most Unwanted Defender by the Player").show(10)
```

Final Output:

- Based on the final production, we can observe the top ten players and their most unwanted defenders.
- Assuming Dirk Nowtizski is the shooter, the most unwanted defender is Hickson, JJ.

```
[Stage 16:=============================================>       (171 + 4) / 200]
+------------------+-----------------------------------+
|Name of the Player|Most Unwanted Defender by the Player|
+------------------+-----------------------------------+
|    dirk nowtizski|                        Hickson, JJ|
|      andre miller|                     Splitter, Tiago|
|    jamal crawford|                          Lawson, Ty|
|       joe johnson|                       Gordon, Aaron|
|       chris kaman|                    Whiteside, Hassan|
|        boris diaw|                         Vonleh, Noah|
|  kendrick perkins|                       Garnett, Kevin|
|   leandro barbosa|                        Harden, James|
|     zaza pachulia|                         Noah, Joakim|
|       mo williams|                         Scola, Luis|
+------------------+-----------------------------------+
only showing top 10 rows
```

Answer 1: The data analysis displays **Each Player's "Most Unwanted Defender"**, which is the defender against whom the player has the lowest shooting success (hit rate). For example, when **Dirk Nowtizski** shoots, **Hickson, JJ** is regarded as his most difficult opponent. This indicates Dirk's chances of scoring are greatly reduced when Hickson is the nearest defender, demonstrating Hickson's defensive efficacy against Dirk.

<u>Question 2:</u> For each player, we define the comfortable zone of shooting is a matrix of, {SHOT DIST, CLOSE DEF DIST, SHOT CLOCK} Please develop a Spark-based algorithm to classify each player's records into 4 comfortable zones. Considering the hit rate, which zone is the best for James Harden, Chris Paul, Stephen Curry, and Lebron James.

Spark Session Creation:

```
#Initialized the Spark session and enabled its evaluation for result
nycSpark = pyspark.sql.SparkSession.builder.appName("VaishnaviP_Assignment2_NBAPart2").getOrCreate()
nycSpark.conf.set("spark.sql.repl.eagerEval.enabled", True)
```

```
nycSpark
```

SparkSession - in-memory

SparkContext

Spark UI

Version	v3.1.1
Master	local[*]
AppName	VaishnaviP_Assignment2_NBAPart2

Loading the data:

```
#Define the path to the dataset for NYC Parking Violations for Fiscal Year 2024
DatasetPath = '/home/exouser/Downloads/shot_logs.csv'

#Loading the dataset as a DataFrame named ParkingDataNYC
NBADataset = nycSpark.read.format("csv").option("header", "true").option("inferSchema", "true").load(DatasetPath)
```

```
NBADataset.show()
```

```
+--------+--------------------+--------+---+-----------+-----------+------+---------+---------+--------+---------+---------+--------+
----------+----------------+-------------------------+-------------+---+---+------------+---------+
| GAME_ID|             MATCHUP|LOCATION|  W|FINAL_MARGIN|SHOT_NUMBER|PERIOD|GAME_CLOCK|SHOT_CLOCK|DRIBBLES|TOUCH_TIME|SHOT_DIST|PTS_TYPE|SHOT
_RESULT| CLOSEST_DEFENDER|CLOSEST_DEFENDER_PLAYER_ID|CLOSE_DEF_DIST|FGM|PTS|  player_name|player_id|
+--------+--------------------+--------+---+-----------+-----------+------+---------+---------+--------+---------+---------+--------+
----------+----------------+-------------------------+-------------+---+---+------------+---------+
|21400899|MAR 04, 2015 - CH...|       A|  W|         24|          1|     1|     1:09|     10.8|       2|      1.9|      7.7|       2|
made|   Anderson, Alan|                   101187|          1.3|  1|  2|brian roberts|   203148|
|21400899|MAR 04, 2015 - CH...|       A|  W|         24|          2|     1|     0:14|      3.4|       0|      0.8|     28.2|       3|
missed|Bogdanovic, Bojan|                   202711|          6.1|  0|  0|brian roberts|   203148|
|21400899|MAR 04, 2015 - CH...|       A|  W|         24|          3|     1|     0:00|     null|       3|      2.7|     10.1|       2|
missed|Bogdanovic, Bojan|                   202711|          0.9|  0|  0|brian roberts|   203148|
|21400899|MAR 04, 2015 - CH...|       A|  W|         24|          4|     2|    11:47|     10.3|       2|      1.9|     17.2|       2|
missed|    Brown, Markel|                   203900|          3.4|  0|  0|brian roberts|   203148|
|21400899|MAR 04, 2015 - CH...|       A|  W|         24|          5|     2|    10:34|     10.9|       2|      2.7|      3.7|       2|
missed|  Young, Thaddeus|                   201152|          1.1|  0|  0|brian roberts|   203148|
|21400899|MAR 04, 2015 - CH...|       A|  W|         24|          6|     2|     8:15|      9.1|       2|      4.4|     18.4|       2|
missed|  Williams, Deron|                   101114|          2.6|  0|  0|brian roberts|   203148|
|21400899|MAR 04, 2015 - CH...|       A|  W|         24|          7|     4|    10:15|     14.5|      11|      9.0|     20.7|       2|
missed|    Jack, Jarrett|                   101127|          6.1|  0|  0|brian roberts|   203148|
|21400899|MAR 04, 2015 - CH...|       A|  W|         24|          8|     4|     8:00|      3.4|       3|      2.5|      3.5|       2|
made|   Plumlee, Mason|                    203486|          2.1|  1|  2|brian roberts|   203148|
|21400899|MAR 04, 2015 - CH...|       A|  W|         24|          9|     4|     5:14|     12.4|       0|      0.8|     24.6|       3|
missed|   Morris, Darius|                   202721|          7.3|  0|  0|brian roberts|   203148|
|21400890|MAR 03, 2015 - CH...|       H|  W|          1|          1|     2|    11:32|     17.4|       0|      1.1|     22.4|       3|
missed| Ellington, Wayne|                   201961|         19.8|  0|  0|brian roberts|   203148|
|21400890|MAR 03, 2015 - CH...|       H|  W|          1|          2|     2|     6:30|     16.0|       8|      7.5|     24.5|       3|
missed|      Lin, Jeremy|                   202391|          4.7|  0|  0|brian roberts|   203148|
|21400890|MAR 03, 2015 - CH...|       H|  W|          1|          3|     4|    11:32|     12.1|      14|     11.9|     14.6|       2|
made|      Lin, Jeremy|                    202391|          1.8|  1|  2|brian roberts|   203148|
|21400890|MAR 03, 2015 - CH...|       H|  W|          1|          4|     4|     8:55|      4.3|       2|      2.9|      5.9|       2|
```

Approach:

1. Converting the values to Float

```
#If the shot is scored, the score will be stored as 1 & 0 will be stored if shot is missed.

NBADataset = NBADataset.na.drop()
NBADataset = NBADataset.withColumn(
    'SHOT_RESULT',
    pyspkfnt.when(pyspkfnt.col('SHOT_RESULT') == 'made', 1).otherwise(0).cast('float')
)
```

```
# Check the result if dropped
NBADataset.select("player_name", "SHOT_RESULT").show(5)
```

```
+-------------+-----------+
|  player_name|SHOT_RESULT|
+-------------+-----------+
|brian roberts|        1.0|
|brian roberts|        0.0|
|brian roberts|        0.0|
|brian roberts|        0.0|
|brian roberts|        0.0|
+-------------+-----------+
only showing top 5 rows
```

2. The algorithm picks specified columns ("player_name," "SHOT_DIST," "CLOSE_DEF_DIST," "SHOT_CLOCK," and "SHOT_RESULT") and removes rows with missing data. The "SHOT_RESULT" column is converted to a binary float (1 for'made' shots and 0 for'missed'). The list "features_toCast " is constructed with columns for subsequent analysis, including shot distance, defender distance, and shot clock time.

```
from functools import reduce
features_toCast = ["SHOT_DIST", "CLOSE_DEF_DIST", "SHOT_CLOCK"]

# Uses reduce to apply cast to float operation to each column
NBADataset = reduce(lambda df, col: df.withColumn(col, df[col].cast("float")),features_toCast, NBADataset)
NBADataset.select("SHOT_DIST", "CLOSE_DEF_DIST", "SHOT_CLOCK").show()
```

```
+---------+-------------+----------+
|SHOT_DIST|CLOSE_DEF_DIST|SHOT_CLOCK|
+---------+-------------+----------+
|      7.7|          1.3|      10.8|
|     28.2|          6.1|       3.4|
|     17.2|          3.4|      10.3|
|      3.7|          1.1|      10.9|
|     18.4|          2.6|       9.1|
|     20.7|          6.1|      14.5|
|      3.5|          2.1|       3.4|
|     24.6|          7.3|      12.4|
|     22.4|         19.8|      17.4|
|     24.5|          4.7|      16.0|
|     14.6|          1.8|      12.1|
|      5.9|          5.4|       4.3|
|     26.4|          4.4|       4.4|
|     22.8|          5.3|       6.8|
|     24.7|          5.6|       6.4|
|     25.0|          5.4|      17.6|
|     25.6|          5.1|       8.7|
|     24.2|         11.1|      20.8|
|     25.4|          3.5|      17.5|
|     19.1|          4.0|      19.5|
+---------+-------------+----------+
only showing top 20 rows
```

```
]: from pyspark.sql.functions import col, sum, when

# Assuming NBADataset is the initial DataFrame
NBAshotresult = NBADataset.groupBy(
    col("player_id").alias("ID of Player"),
    col("CLOSEST_DEFENDER_PLAYER_ID").alias("ID of Defender")
).agg(
    sum(when(col("SHOT_RESULT") == "made", 1).otherwise(0)).alias("Scored a Shot"),
    sum(when(col("SHOT_RESULT") == "missed", 1).otherwise(0)).alias("Not Scored a Shot")
)

NBAshotresult.show()
```

```
+------------+--------------+-------------+-----------------+
|ID of Player|ID of Defender|Scored a Shot|Not Scored a Shot|
+------------+--------------+-------------+-----------------+
|      203148|        101179|            0|                1|
|      202687|        201980|            1|                0|
|        2744|          1717|            0|                2|
|      203469|        202329|            1|                1|
|      201945|        202322|            0|                3|
|      202689|        202699|            6|                8|
|      202689|        203924|            1|                0|
|      203077|          2730|            1|                0|
|      203077|        201584|            2|                0|
|      202362|        201188|            2|                0|
|      202330|        201978|            2|                1|
|      202324|        203135|            1|                2|
|      203957|          2617|            1|                0|
|        2430|        203092|            3|                2|
|        2430|        200770|            1|                0|
|      202391|        202328|            0|                1|
|      101179|        202390|            0|                1|
|      201961|        203200|            1|                0|
|      202325|        203498|            1|                4|
|      203135|        201951|            1|                0|
+------------+--------------+-------------+-----------------+
only showing top 20 rows
```

3. VAssemblerUsed combines the supplied features (features_toCast) into a single "features_toCast" column for each player in the NBADataset DataFrame.

```
#Creates a feature vector
VAssemblerUsed = VectorAssembler(inputCols=features_toCast, outputCol="features_toCast")

NBADatasetvct = VAssemblerUsed.transform(NBADataset).select('player_name', 'features_toCast', col('SHOT_RESULT').cast('float').alias('SHOT_RES
NBADatasetvct.show(6)
```

```
+-------------+--------------------+-----------+
|  player_name|     features_toCast|SHOT_RESULT|
+-------------+--------------------+-----------+
|brian roberts|[7.69999980926513...|        1.0|
|brian roberts|[28.2000007629394...|        0.0|
|brian roberts|[17.2000007629394...|        0.0|
|brian roberts|[3.70000004768371...|        0.0|
|brian roberts|[18.3999996185302...|        0.0|
|brian roberts|[20.7000007629394...|        0.0|
+-------------+--------------------+-----------+
only showing top 6 rows
```

4. The algorithm creates ClusterModel by initializing a K-Means clustering model with k=4 clusters and fitting it to changed data. It filters player data for specific players

(e.g., 'James Harden', 'Chris Paul', 'Stephen Curry', and 'Lebron James'). The fitted K-Means model predicts cluster assignments for selected players, producing a data frame with player names, predicted cluster labels, and shot results.

```
import warnings
from pyspark.ml.clustering import KMeans
from pyspark.sql.functions import col
warnings.filterwarnings('ignore')

#Fit K-Means clustering with a predetermined number of clusters to the data set.
KmeansCluster = KMeans(k=4, seed=1, featuresCol="features_toCast")
ClusterModel = KmeansCluster.fit(NBADatasetvct)

#Players specified in the question
PlayersinQuest = ['james harden', 'chris paul', 'stephen curry', 'lebron james']
NBAPlayersData = NBADatasetvct.filter(col('player_name').isin(PlayersinQuest))
```

5. Transforming the Player's Data using the fitted Model

```
# Transforming the Player's_Data using the fitted model
NBADataTransformed = ClusterModel.transform(NBAPlayersData).select('player_name', 'SHOT_RESULT', 'prediction')
NBADataTransformed.show()
```

```
+-------------+-----------+----------+
|  player_name|SHOT_RESULT|prediction|
+-------------+-----------+----------+
|stephen curry|        0.0|         3|
|stephen curry|        0.0|         1|
|stephen curry|        1.0|         2|
|stephen curry|        1.0|         1|
|stephen curry|        0.0|         2|
|stephen curry|        0.0|         0|
|stephen curry|        0.0|         1|
|stephen curry|        0.0|         1|
|stephen curry|        0.0|         1|
|stephen curry|        1.0|         1|
|stephen curry|        1.0|         1|
|stephen curry|        1.0|         1|
|stephen curry|        0.0|         1|
|stephen curry|        0.0|         3|
|stephen curry|        1.0|         1|
|stephen curry|        0.0|         1|
|stephen curry|        0.0|         1|
|stephen curry|        1.0|         1|
|stephen curry|        0.0|         1|
|stephen curry|        0.0|         1|
+-------------+-----------+----------+
only showing top 20 rows
```

6. The code below calculates the average shot outcome for each player, predicts clusters, and orders results. The algorithm calculates the highest average shot outcome for each player and uses a join to identify the rows with the highest averages. The bestZone DataFrame preserves the original DataFrame's columns and displays the results.

```
#Calculates average_Shot results grouped by player and their prediction
sql_query = """
SELECT player_name, prediction, AVG(SHOT_RESULT) AS Avgshot
FROM player_zones
GROUP BY player_name, prediction
ORDER BY player_name, prediction
"""
Res_AvgShot = nycSpark.sql(sql_query)

# Find the maximum average shot result for each player
Res_MaxAvg = Res_AvgShot.groupBy("player_name").agg(
    pyspkfnt.max("Avgshot").alias("Res_MaxAvg")
)

# Join to find the best shooting zone per player where their average shot result is the maximum
Bestzones = Res_AvgShot.alias("a").join(
    Res_MaxAvg.alias("b"),
    (pyspkfnt.col("a.player_name") == pyspkfnt.col("b.player_name")) &
    (pyspkfnt.col("a.Avgshot") == pyspkfnt.col("b.Res_MaxAvg"))
).select("a.Avgshot", "a.player_name", "a.prediction")

# Display the optimal zones
Bestzones.show()
```

Final Output:

- In the 'prediction' column, Zone-1 has a prediction value of 0, Zone-2 has one, Zone-3 has two, and Zone 4 has three predictions.
- To evaluate each player's comfort zone, we divided the data by player and zone, computing the average score for each group.
- The table shows that LeBron, James, and Stephen have the greatest Zone 1, while Chris has the best Zone 4.

```
+------------------+-------------+----------+
|           Avgshot|  player_name|prediction|
+------------------+-------------+----------+
|0.5604395604395604| james harden|         3|
|0.5563380281690141|   chris paul|         0|
|0.6613545816733067| lebron james|         3|
|0.6350710900473934|stephen curry|         3|
+------------------+-------------+----------+
```

Answer 2: The analysis identifies the best shooting zones for notable NBA players based on hit rates. **Chris Paul** excels in **Zone 0 with a 56%** hit rate. **LeBron James, Stephen Curry, and James Harden** perform optimally in **Zone 3**, boasting hit rates of **66.13%, 63.50%, and 56.04% respectively**. This highlights zone 3 as generally the most advantageous, except for Chris Paul.

References:

Based on the video provided by TA:

- **installation of Hadoop on Jetstream2 Ubuntu instance with streaming example.mp4**
- **installation of single-node Spark on Jetstream2 instance.mp4**